

NAKAST

Manual de Usuario

UNIVERSIDAD DE VALPARAÍSO

Autores:

JAVIER ALEJANDRO TOGNARELLI SANTIAGO

DANIEL FERNANDO ESCOBAR ARAYA

FERNANDO ANDRÉS AMAYA INZUNZA

SEPTIEMBRE 2026

Tabla de Contenidos

1. Introducción
2. Acceso e instalación
 - 2.1 Requisitos
 - 2.2 Descarga
 - 2.3 Verificación de la instalación
 - 2.4 Clave de acceso a PubMLST
 - 2.5 Ejecución sin Conda
3. Módulo de análisis: uso del pipeline
 - 3.1 Un ejemplo completo
 - 3.2 Especificación de entrada
 - 3.3 Referencia de parámetros
 - 3.4 Referencia de salidas
 - 3.5 Cómo interpretar los resultados
 - 3.6 Evaluación de la calidad de los datos
4. Módulo técnico: administración y mantenimiento
 - 4.1 Arquitectura del flujo
 - 4.2 Cómo se llaman los alelos
 - 4.3 Asignación de ST y CC
 - 4.4 Especies soportadas
 - 4.5 Supuestos
 - 4.6 Limitaciones
 - 4.7 Reproducibilidad
 - 4.8 Notas de diseño
5. Soporte, citación y licencia
 - 5.1 Reportar problemas
 - 5.2 Autoría
 - 5.3 Cómo citar NAKAST
 - 5.4 Referencias de los componentes
 - 5.5 Componentes de terceros y fuente de datos
6. Preguntas frecuentes

An English version of this manual is available at [USER_MANUAL .md](#).

1. Introducción

NAKAST (*Nanopore Amplicon KMA Allele Sequence Typing*) es un pipeline desarrollado en Nextflow que determina el perfil MLST de aislados bacterianos a partir de lecturas de secuenciación de amplicones Oxford Nanopore (ONT). A partir de los archivos FASTQ de cada muestra y del nombre de su especie, entrega una tabla con el Sequence Type (ST) y, cuando el esquema lo define, el Complejo Clonal (CC) de cada aislado.

MLST (*Multi Locus Sequence Typing*) caracteriza un aislado mediante la secuencia de un conjunto pequeño de genes *housekeeping*, habitualmente siete. Cada secuencia distinta de un locus recibe un **número de alelo**; el conjunto ordenado de estos números forma el **perfil alélico** (por ejemplo, 9-1-4-1-3-3-2), y cada perfil distinto corresponde a un **Sequence Type**. Un perfil solo coincide con un ST si todos sus alelos coinciden exactamente con alelos conocidos. Los STs emparentados, que comparten la mayoría de sus alelos, se agrupan en **complejos clonales**.

Las definiciones de alelos y perfiles provienen de **PubMLST**, que NAKAST consulta en cada corrida para obtener el esquema vigente de la especie. Las lecturas se alinean contra esos alelos con **KMA**, un alineador diseñado para bases de datos redundantes como las de MLST, capaz de distinguir entre alelos que difieren en unas pocas bases.

El pipeline funciona con cualquiera de los 135 esquemas MLST de PubMLST que definen perfiles ST. La especie se declara por muestra, de modo que una misma corrida puede incluir varias especies: la base de datos se descarga una vez por especie y se genera un reporte por cada una.

La Figura 1 resume el flujo de trabajo, desde la validación de la tabla de muestras hasta la generación del reporte final.

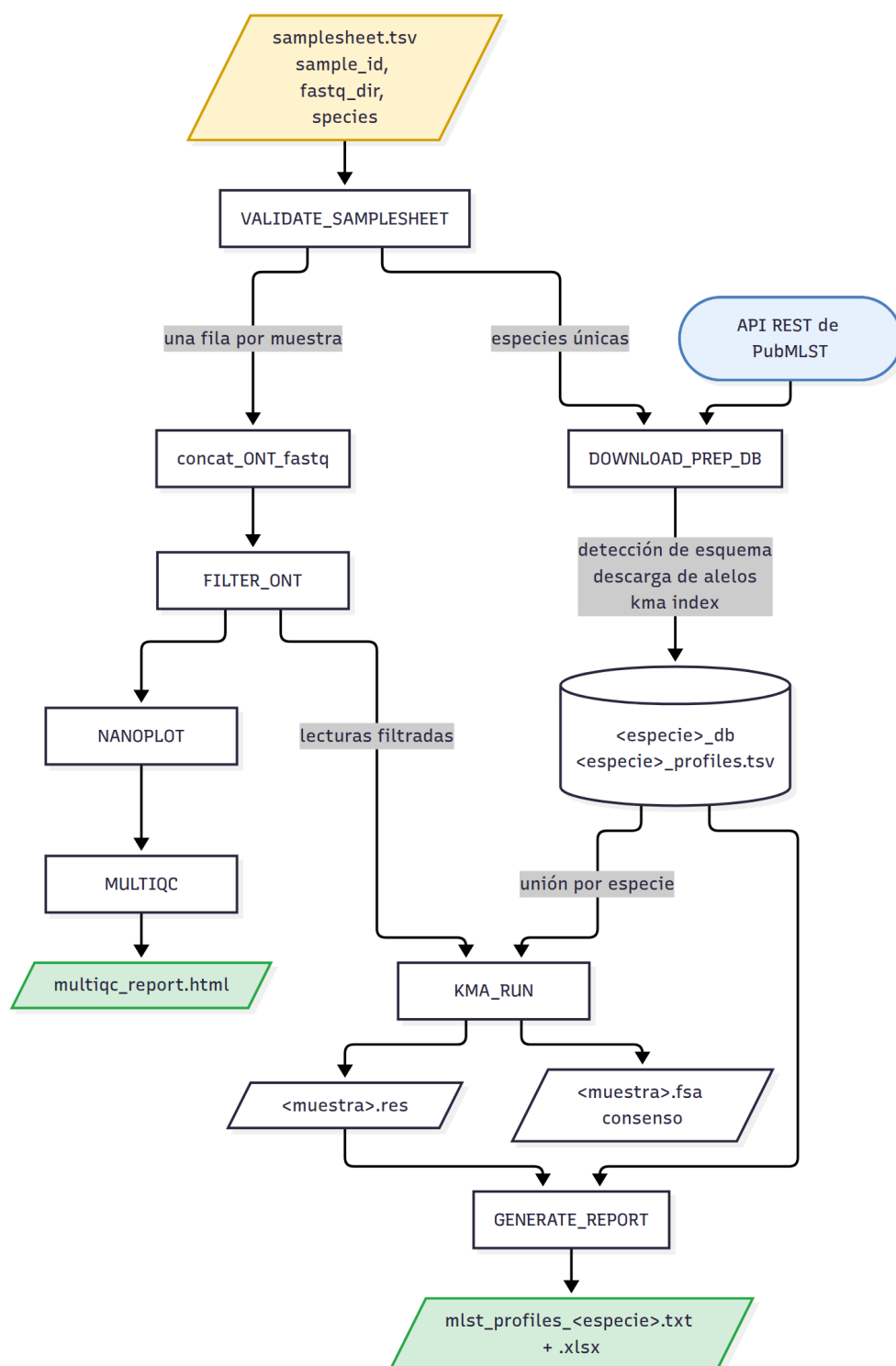


Figura 1. Flujo de trabajo de NAKAST. En amarillo la entrada, en azul la fuente externa de datos y en verde los resultados finales.

2. Acceso e instalación

NAKAST no requiere registro ni cuenta de usuario. Se distribuye como código fuente a través de su repositorio en GitHub y se ejecuta localmente desde la línea de comandos.

2.1 Requisitos

Componente	Versión	Observaciones
Sistema operativo	Linux	Plataforma en la que fue desarrollado y probado
Nextflow	25.10 o posterior	Probado en 25.10.2 y 26.04.6
Conda	>= 26.1.1	Miniconda o Miniforge
Mamba	>= 2.4.0	Acelera la construcción de los ambientes
Acceso a internet	—	PubMLST se consulta en cada corrida
Cuenta en PubMLST	—	Gratuita; su clave de acceso da el esquema completo (sección 2.4)

No es necesario instalar KMA, filtlong, chopper, NanoPlot ni MultiQC: cada proceso declara su propio ambiente Conda con versiones fijas, y Nextflow los construye en la primera ejecución.

2.2 Descarga

```
git clone https://github.com/javiertognarelli/nakast.git
cd nakast
```

2.3 Verificación de la instalación

```
nextflow run nakast.nf --help
nextflow config .           # debe mostrar conda { enabled = true; useMamba = true }
```

La primera corrida real construye los ambientes Conda, lo que agrega algunos minutos. Quedan almacenados en `$HOME/.nextflow/conda` y se reutilizan en las corridas siguientes.

2.4 Clave de acceso a PubMLST

Desde 2025, PubMLST entrega sin autenticación solo los datos depositados hasta el 31 de diciembre de 2024. Los alelos y STs definidos después requieren una cuenta. Por eso NAKAST exige por defecto una **clave de acceso a datos** (*data access key*) de PubMLST, que envía en cada consulta.

Para obtenerla, ingrese a su cuenta en <https://pubmlst.org> y cree una clave de acceso a datos. Luego guárdela una sola vez en el almacén de secretos de Nextflow:

```
read -rs -p "PubMLST API key: " K && nextflow secrets set PUBMLST_API_KEY "$K" &&
unset K
```

El comando pide la clave sin mostrarla y sin dejarla en el historial de la terminal. Nextflow la guarda en `$HOME/.nextflow/secrets/`, en un archivo que solo su usuario puede leer, y

la entrega al proceso de descarga sin escribirla en el directorio de trabajo, en los logs ni en los resultados. Para confirmar que quedó guardada:

```
nextflow secrets list          # debe mostrar PUBMLST_API_KEY
```

La clave es personal: cada usuario debe usar la suya y nunca incluirla en el repositorio ni en archivos de configuración.

Si no tiene cuenta, puede correr igual agregando `--pubmlst_anonymous`. En ese modo NAKAST usa solo los datos depositados hasta el 31 de diciembre de 2024, y cualquier aislado de un ST o alelo definido después aparecerá como no tipificado.

2.5 Ejecución sin Conda

Quien prefiera administrar las herramientas por su cuenta puede usar el perfil `host`, que desactiva Conda y utiliza lo disponible en el `PATH`:

```
nextflow run nakast.nf -profile host --samplesheet samplesheet.tsv
```

En ese caso deben estar instalados `kma`, `filtlong`, `chopper`, `NanoPlot`, `multiqc` y `Python` con `pandas`, `numpy` y `openpyxl`. No es el modo recomendado, porque renuncia a la reproducibilidad que garantizan los ambientes con versiones fijas.

3. Módulo de análisis: uso del pipeline

Esta sección está dirigida a quien ejecuta el pipeline e interpreta sus resultados.

3.1 Un ejemplo completo

Una corrida de tres aislados de *Streptococcus agalactiae*, desde los datos crudos hasta la interpretación.

Paso 1 — revisar los datos de entrada

Sus lecturas ya basecalled y demultiplexadas, un directorio por muestra:

```
ls ~/corrida_2026_03/fastq_pass/
# barcode01 barcode02 barcode03
ls ~/corrida_2026_03/fastq_pass/barcode01/ | head -3
# FBD86797_pass_barcode01_0.fastq.gz
```

Paso 2 — encontrar el nombre de la especie

```
nextflow run nakast.nf --list-species | grep -i agalactiae
# sagalactiae    1    7  yes Streptococcus agalactiae
```

Paso 3 — escribir el samplesheet

Tres columnas separadas por tabulación:

```
printf 'sample_id\tfastq_dir\tspecies\n' > samplesheet.tsv
printf 'SGB11\t%s/barcode01\tsagalactiae\n' ~/corrida_2026_03/fastq_pass >>
    samplesheet.tsv
printf 'SGB12\t%s/barcode02\tsagalactiae\n' ~/corrida_2026_03/fastq_pass >>
    samplesheet.tsv
printf 'SGB13\t%s/barcode03\tsagalactiae\n' ~/corrida_2026_03/fastq_pass >>
    samplesheet.tsv
```

Use rutas absolutas. Las rutas relativas se resuelven contra el directorio desde el que se lanza la corrida.

Paso 4 — ejecutar

```
nextflow run /ruta/a/nakast/nakast.nf \
    --samplesheet samplesheet.tsv \
    --outdir resultados_corrida2026_03
```

Espere entre 5 y 10 minutos para tres muestras en un notebook, la mayor parte en KMA. La primera corrida suma unos minutos extra para construir los ambientes Conda.

Paso 5 — leer el reporte

Sample	ST	CC	adhP	pheS	atr	glnA	sdhA	glcK	tkf	Profile
SGB11	10	cc12	9	1	4	1	3	3	2	OK
SGB12	24	cc452	5	4	4	3	2	3	3	OK
SGB13	2	cc1	1	1	3	1	1	2	2	OK

Tres aislados tipificados: ST10, ST24 y ST2. La sección 3.5 explica qué hacer cuando una fila no se parece a estas.

Paso 6 — revisar el control de calidad

Abra `resultados_corrida2026_03/qc/multiqc_report.html` para confirmar que los conteos de lecturas y las distribuciones de calidad son los esperados para la corrida.

3.2 Especificación de entrada

Un archivo separado por tabulaciones, con fila de encabezado y tres columnas.

Columna	Requerida	Descripción
sample_id	sí	Identificador único. Se usa como prefijo de los archivos de salida
fastq_dir	sí	Directorio con archivos .fastq.gz o .fastq
species	sí	Nombre corto del catálogo, por ejemplo sagalactiae

Las columnas adicionales se ignoran, de modo que los samplesheets escritos para versiones anteriores siguen funcionando.

La validación corre antes de cualquier trabajo pesado y verifica que existan las tres columnas, que `sample_id` no esté vacío y sea único, que los directorios referenciados existan en disco, y que `species` figure en `assets/pubmlst_species.tsv`. Una especie mal escrita falla en segundos con una sugerencia, en vez de hacerlo tras una descarga parcial.

Un lote de especies mezcladas se escribe exactamente igual:

```
sample_id  fastq_dir  species
SGB11     /datos/corrida1/barcode01  sagalactiae
SAL07     /datos/corrida1/barcode02  salmonella
```

3.3 Referencia de parámetros

Entrada y salida

Parámetro	Por defecto	Descripción
<code>--samplesheet</code>	<code>samplesheet.tsv</code>	Tabla de entrada
<code>--outdir</code>	<code>results</code>	Directorio base de salida
<code>--qc_outdir</code>	<code><outdir>/qc</code>	Salidas de control de calidad
<code>--consensus_outdir</code>	<code><outdir>/consensus_sequences</code>	FASTA consenso
<code>--report_outdir</code>	<code><outdir>/reports</code>	Archivos <code>.res</code> y tablas de perfiles
<code>--mlst_report</code>	<code>mlst_profiles</code>	Nombre base del reporte final

Filtrado de lecturas

Parámetro	Por defecto	Descripción
<code>--nano_base_quality</code>	<code>20</code>	Calidad Phred mínima, entregada a chopper
<code>--min_length</code>	<code>300</code>	Longitud mínima de lectura (filtlong)
<code>--keep_percent</code>	<code>80</code>	Conservar este porcentaje de las mejores lecturas por bases (filtlong)

Llamado de alelos

Parámetro	Por defecto	Descripción
<code>--min_depth</code>	<code>30</code>	Profundidad mínima, aplicada en KMA y de nuevo en el script de reporte
<code>--min_identity</code>	<code>90</code>	Identidad mínima respecto del alelo de referencia (%)
<code>--allele_coverage</code>	<code>90</code>	Cobertura mínima del alelo, es decir del molde (%)
<code>--kma_k</code>	<code>31</code>	Tamaño de k-mero para el índice de KMA

Parámetro	Por defecto	Descripción
--generate_consensus	true	Publicar las secuencias consenso por locus

Modos de ejecución

Parámetro	Por defecto	Descripción
--qc_only	false	Solo control de calidad; sin descarga ni tipificación
--list_species	false	Imprimir el catálogo de especies soportadas y salir
--pubmlst_anonymous	false	Consultar PubMLST sin clave; solo datos hasta el 31-12-2024 (sección 2.4)
--help	false	Imprimir la ayuda y salir

Opciones de Nextflow que conviene conocer

Pertenecen a Nextflow y llevan un solo guion.

Opción	Propósito
-resume	Reutilizar el trabajo completado en la corrida anterior
-profile host	Usar las herramientas del PATH en vez de Conda
-with-report	Reporte HTML de ejecución adicional

Recursos

Se asignan por etiqueta en `nextflow.config`. Edite ese archivo según su máquina.

Etiqueta	CPUs	Memoria	Tiempo	Procesos
process_low	2	4 GB	2 h	validación, base de datos, listado de especies
process_medium	6	10 GB	8 h	concatenación, filtrado, QC, reporte
process_high	12	16 GB	24 h	alineamiento con KMA

3.4 Referencia de salidas

```
results/
├── mlst_profiles_<especie>.txt      ← el resultado
├── mlst_profiles_<especie>.xlsx
├── qc/
│   ├── qc_<muestra>/
│   └── multiqc_report.html        ← calidad de la corrida
├── consensus_sequences/
│   └── <muestra>.fsa
├── reports/
│   ├── <muestra>.res              ← evidencia por alelo
│   └── <especie>_profiles.tsv
└── pipeline_info/
```

Qué archivo responde cada pregunta

Pregunta	Archivo
¿Qué ST tiene mi aislado?	mlst_profiles_<especie>.txt
¿Por qué se llamó así este alelo?	reports/<muestra>.res
¿Secuenció bien la corrida?	qc/multiqc_report.html
¿Qué secuencia envió a PubMLST o confirmo por Sanger?	consensus_sequences/<muestra>.fsa
¿Qué versión de PubMLST usó esta corrida?	reports/<especie>_profiles.tsv
¿Cuánto demoró, qué falló?	pipeline_info/

Columnas del reporte final

Columna	Contenido
Sample	Identificador de la muestra
ST	Sequence Type, o - si el perfil no coincide con ninguno
CC	Complejo Clonal, o - si no está asignado o el esquema no lo define
una columna por locus	Llamado del alelo con su anotación de calidad
Profile	OK, o INCOMPLETE (n/N) cuando se recuperaron menos de N loci

Archivos intermedios

- reports/<muestra>.res — salida cruda de KMA. El registro autoritativo de identidad, cobertura, profundidad y puntaje para cada molde considerado.
- reports/<especie>_profiles.tsv — la tabla de perfiles de PubMLST exacta que se usó. Consérvela: PubMLST cambia con el tiempo y este archivo permite reproducir la corrida después.
- consensus_sequences/<muestra>.fsa — consenso de KMA por locus.
- pipeline_info/ — reporte de ejecución, línea de tiempo y traza.

3.5 Cómo interpretar los resultados

Recorra esto en orden cuando una fila no sea un OK limpio con su ST.

Las anotaciones de alelo

Notación	Significado
9	Coincidencia exacta con el alelo 9
~9	El más cercano es el alelo 9, pero la secuencia difiere
9?	Coincide con el alelo 9, pero la lectura es más larga que la referencia
INS	La lectura cubre menos que el alelo completo
-	Sin llamado: bajo los umbrales de identidad, cobertura o profundidad

Solo los enteros sin anotación pueden coincidir con un perfil de PubMLST. Si cualquier locus lleva ~, ?, INS o -, el perfil no puede coincidir y ST será -. La búsqueda del ST no falló; el perfil simplemente estaba incompleto.

Guía de decisión

ST = - pero todos los loci tienen un entero sin anotación. El perfil está completo pero esa combinación no existe en PubMLST. Es un ST genuinamente nuevo. Considere depositarlo.

ST = - y un locus muestra ~N. El caso más frecuente. Ese locus difiere de todos los alelos conocidos. O es un alelo nuevo, o el consenso arrastra un error de secuenciación. Para distinguirlos, revise la profundidad de ese locus en el archivo .res. Una diferencia consistente a alta profundidad apunta a un alelo nuevo real; confírmelo por Sanger usando el consenso de consensus_sequences/. Una diferencia a baja profundidad es más probablemente un artefacto.

ST = - y un locus muestra -. Ese locus no se recuperó. Revise su profundidad en el archivo .res. Una profundidad bajo --min_depth significa que el amplicón rindió mal, lo cual es un problema de laboratorio y no de análisis. Reamplifique o secuencie más profundo.

Profile = INCOMPLETE (6/7). Se recuperaron seis de siete loci. El diagnóstico es el mismo: identifique el locus faltante y revise su profundidad.

ST asignado pero CC = -. Habitualmente es correcto. Muchos STs no tienen complejo clonal, y 33 de los 135 esquemas no definen ninguno. Revise la columna CC del catálogo para su especie.

Todas las muestras con ST = -. Sospeche del valor de species antes que de los datos. Tipificar contra el esquema equivocado produce resultados vacíos sin ningún error.

Cómo leer el archivo de evidencia

```
column -t results/reports/SGB20.res | head
```

Las columnas que importan son #Template (locus y alelo), Template_Identity, Template_Coverage, Query_Coverage y Depth. Un alelo llamado ~9 mostrará Template_Identity bajo 100. Un locus reportado como - - puede estar ausente del archivo por completo, lo que significa que nada pasó los umbrales.

3.6 Evaluación de la calidad de los datos

NAKAST no impone un umbral de calidad más allá de --min_depth. Use estos puntos de referencia, obtenidos en la validación con el panel de amplicones propio para *S. agalactiae*.

Métrica	Habitual	Preocupante
Profundidad por locus	de varios cientos a algunos miles	bajo 100
Loci recuperados	7 de 7	6 o menos
Alelos anotados por muestra	0	2 o más
Calidad mediana de lectura	alrededor de Q18	bajo Q12

La profundidad varía bastante entre loci en un panel de amplicones: una diferencia de diez veces entre el mejor y el peor amplicón es normal. Lo que importa es que el locus más débil supere --min_depth.

Una profundidad muy alta no rescata una amplificación deficiente. Si un locus es consistentemente débil en todas las muestras de una corrida, la causa está en el partidor o en la reacción, no en el análisis.

4. Módulo técnico: administración y mantenimiento

Esta sección está dirigida a quien administra, adapta o mantiene el pipeline. Describe cómo funciona internamente, qué supone sobre los datos y dónde están sus límites.

4.1 Arquitectura del flujo

Proceso	Herramienta	Propósito
VALIDATE_SAMPLESHEET	python, pandas	Verificación previa del samplesheet
DOWNLOAD_PREP_DB	python, kma	Detectar esquema, descargar alelos y perfiles, indexar
concat_ONT_fastq	coreutils, gzip	Unir un directorio de barcode en un solo FASTQ
FILTER_ONT	filtlong, chopper	Filtrado por longitud y calidad
NANOPLLOT	NanoPlot	Control de calidad por muestra
MULTIQC	MultiQC	Reporte de calidad agregado

Proceso	Herramienta	Propósito
KMA_RUN	kma	Alinear lecturas contra la base de alelos
GENERATE_REPORT	python, pandas, openpyxl	Llamado de alelos, asignación de ST y CC, reportes
LIST_SPECIES	python	Imprimir el catálogo de especies soportadas

El comportamiento ante fallas difiere por etapa, de forma deliberada:

Proceso	Estrategia	Razón
VALIDATE_SAMPLESHEET	terminate	Un samplesheet inválido debe detener todo de inmediato
DOWNLOAD_PREP_DB	3 reintentos, luego finish	Sobrevive cortes de red transitorios; deja terminar el QC para que -resume pueda continuar
KMA_RUN, GENERATE_REPORT	terminate	Una falla silenciosa aquí produciría una corrida sin resultados y exit 0
concat_ONT_fastq, FILTER_ONT, NANOPLOT, MULTIQC	ignore	Una muestra mala no debe abortar el lote

4.2 Cómo se llaman los alelos

Paso 1 — filtro de profundidad

Los moldes bajo `--min_depth` se descartan, tanto en KMA como nuevamente en el script de reporte.

Paso 2 — elegir un ganador por locus

Varios alelos del mismo locus suelen atraer lecturas, ya que difieren en unas pocas bases. NAKAST los ordena con un puntaje de confianza:

$$\text{Score_Ratio} = \text{Score} / \text{Expected}$$

$$\text{Confidence} = \text{Score_Ratio} \times \log_{10}(\text{Depth}) \times \text{bonus_perfecto}$$

`bonus_perfecto` vale 1.5 cuando identidad, cobertura del molde y cobertura de la consulta son todas 100%, y 1.0 en caso contrario. El molde con mayor confianza gana el locus.

El bonus es multiplicativo y no absoluto a propósito: una coincidencia perfecta todavía necesita profundidad que la respalde, de modo que una sola lectura bien alineada no puede superar a un alelo con cobertura profunda.

Paso 3 — anotar el llamado

Notación	Condición
9	Identidad, cobertura del molde y de la consulta, todas 100%
~9	Cobertura de consulta y molde 100%, identidad bajo 100%
9?	Cobertura de consulta sobre 100%, identidad y cobertura sobre los umbrales
INS	Cobertura de consulta bajo 100%
-	Identidad o cobertura del molde bajo los umbrales, o ningún molde pasó

Una nota sobre ?: una cobertura de consulta sobre 100% significa que la lectura es *más larga* que el molde. Con un panel de amplicones esto refleja read-through y no truncamiento, así que el llamado suele ser correcto. Interprete 9? como “probablemente el alelo 9, vale la pena confirmarlo” y no como una falla.

4.3 Asignación de ST y CC

Los llamados de alelo se unen en una firma (9/1/4/1/3/3/2) que se compara contra la misma firma construida a partir de la tabla de perfiles de PubMLST. La comparación es exacta y sobre texto:

- Los loci provienen de <especie>_loci.txt, escrito desde la definición del esquema, y luego se ordenan como aparecen en la tabla de perfiles, de modo que el reporte conserva el orden canónico de PubMLST.
- La tabla de perfiles se lee como texto para que un valor faltante no convierta en silencio una columna de locus en números decimales y produzca 1.0/1/2/..., lo que haría fallar todas las comparaciones.
- La columna de CC se detecta de forma tolerante (clonal_complex, clonal complex, cc, clonalcomplex). Si no existe, CC = -.
- Si no hay coincidencia se devuelve ST = -. **NAKAST nunca reporta un ST cercano o aproximado.**

4.4 Especies soportadas

assets/pubmlst_species.tsv y docs/supported_species.md catalogan los **135 esquemas de PubMLST** que definen perfiles ST. Para regenerarlos:

```
python3 bin/list_pubmlst_species.py --write-catalogue .
```

Dos supuestos que se cumplen para *S. agalactiae* pero no en general, y que NAKAST por lo tanto resuelve en tiempo de ejecución:

- **El esquema MLST no siempre es el número 1.** Siete esquemas usan otro identificador; *Salmonella* usa el 2. NAKAST selecciona el esquema cuya descripción comienza con MLST.

- **Los esquemas no siempre tienen siete loci.** Van de 2 a 10, y 48 de 135 difieren de siete. La lista de loci proviene de la definición del esquema, nunca de una lista fija.

Los nombres de locus también son menos regulares de lo que parecen: MLST_adk, 16S_rRNA e int_hyp contienen guiones bajos, por lo que los identificadores de alelo se separan por la derecha.

4.5 Supuestos

Estos importan para interpretar los resultados.

1. **Secuenciación de amplicones, no genoma completo.** Se asume que las lecturas provienen de amplificación dirigida de los loci MLST. Se espera profundidad de cientos o miles, y los umbrales por defecto reflejan eso. Datos de genoma completo darán una profundidad por locus mucho menor y muchos llamados -.
2. **La especie se conoce y se declara.** NAKAST no identifica el organismo; tipifica la muestra contra el esquema que usted especifique. Una especie equivocada produce un resultado vacío, no un error.
3. **PubMLST está accesible y es la fuente autoritativa.** La base se descarga fresca en cada corrida, de modo que los resultados dependen del estado de PubMLST en ese momento. Por eso la tabla de perfiles se publica junto con las salidas. El esquema completo solo se obtiene con una clave de acceso válida (sección 2.4).
4. **El esquema define perfiles ST.** Los esquemas sin clave primaria, como cgMLST, quedan fuera del catálogo y se rechazan al momento de la descarga.
5. **Los identificadores de muestra son únicos** en todo el samplesheet. Dan nombre a los archivos de salida.
6. **Las lecturas están basecalled y demultiplexadas.** NAKAST parte de FASTQ por barcode. No hace basecalling, ni demultiplexado, ni recorte de adaptadores y barcodes.
7. **Un organismo por muestra.** Cultivos mixtos o contaminados producen alelos en competencia en algunos loci; el puntaje de confianza elegirá uno, y una anotación ~ suele ser la única señal visible.

4.6 Limitaciones

1. **Solo Oxford Nanopore.** No hay ruta para lecturas cortas. Los parámetros de filtrado y alineamiento están afinados para el perfil de error de ONT, y el samplesheet recibe un directorio de archivos FASTQ y no pares de lecturas.
2. **No detecta ni deposita alelos nuevos.** Una secuencia ausente de PubMLST se reporta como ~N contra el alelo conocido más cercano. NAKAST no la marcará como nueva, no propondrá un número ni preparará un depósito. La confirmación, por Sanger por ejemplo, es un paso manual; el consenso en consensus_sequences/ es el punto de partida.
3. **Requiere red.** No hay modo sin conexión ni caché local entre corridas. Cada corrida vuelve a descargar el esquema.
4. **Datos incompletos sin clave.** Con --pubmlst_anonymous solo se usan datos depositados hasta el 31 de diciembre de 2024. En *S. agalactiae*, en septiembre de 2026, eso dejaba fuera 301 de 2673 STs (11%).

5. **Condiciones de uso de los datos recientes.** Los datos de PubMLST depositados desde 2025, que se obtienen con la clave, solo pueden usarse para investigación académica no comercial o para vigilancia en salud pública, y no pueden redistribuirse. Su uso comercial requiere una licencia de la Universidad de Oxford. Los datos anteriores a 2025 no tienen esa restricción.
6. **Validado biológicamente en una sola especie.** La arquitectura es agnóstica de especie y se ejercitó contra los esquemas de *Salmonella*, *Brucella*, *Vibrio* y *Mycoplasma genitalium*, pero la validación contra aislados conocidos se hizo solo para *S. agalactiae* con el panel de amplicones propio. Los parámetros de alineamiento están afinados para ese panel.
7. **Los complejos clonales dependen del esquema.** 33 de 135 esquemas no definen ninguno, y aun donde existen, muchos STs no tienen CC.
8. **No detecta contaminación ni muestras mixtas.** No hay una verificación explícita de alelos múltiples en un mismo locus.
9. **Los umbrales de profundidad son globales.** --min_depth se aplica por igual a todos los loci. Un panel donde un amplicón rinde sistemáticamente mal puede requerir bajar el umbral para toda la corrida.
10. **Un reporte por especie.** Los lotes que abarcan varias especies producen varios archivos en vez de una tabla única, porque las columnas de loci difieren.

4.7 Reproducibilidad

Cada proceso declara su propio ambiente Conda con versiones exactas:

Proceso	Ambiente
VALIDATE_SAMPLESHEET	python=3.13.11 pandas=2.3.3
DOWNLOAD_PREP_DB, KMA_RUN	python=3.13.11 kma=1.6.8
concat_ONT_fastq	python=3.13.11 coreutils=9.5 gzip=1.12
FILTER_ONT	python=3.13.11 filtlong=0.3.1 chopper=0.12.0 gzip=1.12
NANOPLLOT	python=3.13.11 nanoplot=1.46.2
MULTIQC	python=3.13.11 multiqc=1.33
GENERATE_REPORT	python=3.13.11 pandas=2.3.3 numpy=2.4.0 openpyxl=3.1.5
LIST_SPECIES	python=3.13.11

Los ambientes se cachean en `$HOME/.nextflow/conda` y se construyen con mamba. Las especificaciones idénticas comparten un mismo ambiente.

La única entrada que *no* está fijada es PubMLST, que cambia a medida que los curadores agregan alelos y STs. Para reproducir una corrida pasada, conserve `reports/<especie>_profiles.tsv` y la traza de `pipeline_info/` junto al reporte.

4.8 Notas de diseño

Se registran aquí dos hallazgos del ajuste contra datos reales de *S. agalactiae*, porque son contraintuitivos y fáciles de reintroducir.

El -mrc de KMA no es cobertura del alelo. -mrc es *minimum query coverage*: la fracción de la **lectura** que debe alinear. Una versión anterior le pasaba --allele_coverage. Con lecturas de alrededor de 950 pb contra amplicones de 500 pb, la mayoría de las lecturas no puede alcanzar 90% de cobertura de consulta. Los loci con lecturas abundantes sobrevivían igual, pero los loci poco amplificados perdían todas sus lecturas y desaparecían pese a ser coincidencias perfectas al 100% de identidad y cobertura. El flag se eliminó; la cobertura del alelo se controla aguas abajo sobre Template_Coverage, que es donde corresponde.

Los umbrales de calidad de filtlong no están en escala Phred. --min_mean_q y --min_window_q toman un porcentaje de exactitud de 0 a 100, así que pasarles 20 no filtra absolutamente nada (Q20 corresponde a 99). Fijarlos en un valor que sí filtre resultó perjudicial: en 95 los loci poco amplificados perdieron todo su soporte, porque los loci de baja abundancia también tienden a llevar las lecturas de peor calidad. Los flags se eliminaron y la calidad se controla únicamente con chopper; en escala Phred, mediante --nano_base_quality.

Un resultado negativo relacionado: escalonar umbrales de calidad (Q30 → Q25 → Q20) no es viable con datos típicos de amplicones ONT. Con una calidad mediana por lectura cercana a Q18, los umbrales sobre Q22 aproximadamente retienen una fracción de un por ciento de las bases, y los fragmentos sobrevivientes son más cortos que un amplicón, por lo que no pueden cubrir un alelo.

5. Soporte, citación y licencia

5.1 Reportar problemas

Abra un issue en <https://github.com/javiertognarelli/nakast/issues>. Incluya la versión de Nextflow (nextflow -version), el comando que ejecutó, y la parte relevante de .nextflow.log.

5.2 Autoría

Javier Alejandro Tognarelli Santiago — Genómica UV, Escuela de Medicina, Universidad de Valparaíso, Chile

Daniel Fernando Escobar Araya — Unidad de Investigación e Innovación, Instituto de Salud Pública, Chile

Fernando Andrés Amaya Inzunza — Unidad de Investigación e Innovación, Instituto de Salud Pública, Chile

5.3 Cómo citar NAKAST

Copie la forma que corresponda al estilo de su documento.

Estilo Vancouver

Tognarelli Santiago JA, Escobar Araya DF, Amaya Inzunza FA. NAKAST: MLST typing from Oxford Nanopore amplicon sequencing [software]. Versión 1.0. Valparaíso:

Universidad de Valparaíso; 2026. Disponible en: <https://github.com/javiertognarelli/nakast>

Estilo APA (7.^a edición)

Tognarelli Santiago, J. A., Escobar Araya, D. F., & Amaya Inzunza, F. A. (2026). *NAKAST: MLST typing from Oxford Nanopore amplicon sequencing* (Versión 1.0) [Software]. Universidad de Valparaíso. <https://github.com/javiertognarelli/nakast>

BibTeX, para gestores de referencias y LaTeX:

```
@software{nakast2026,  
  author    = {Tognarelli Santiago, Javier Alejandro and  
              Escobar Araya, Daniel Fernando and  
              Amaya Inzunza, Fernando Andrés},  
  title     = {{NAKAST}: {MLST} typing from {Oxford Nanopore} amplicon sequencing},  
  version   = {1.0},  
  year      = {2026},  
  publisher = {Universidad de Valparaíso},  
  url       = {https://github.com/javiertognarelli/nakast}  
}
```

5.4 Referencias de los componentes

Una publicación que use resultados de NAKAST debería citar también la fuente de datos y las herramientas en que se apoya. Referencias en estilo Vancouver, listas para copiar:

1. Jolley KA, Bray JE, Maiden MCJ. Open-access bacterial population genomics: BIGSdb software, the PubMLST.org website and their applications. *Wellcome Open Res.* 2018;3:124. doi:10.12688/wellcomeopenres.14826.1
2. Clausen PTL, Aarestrup FM, Lund O. Rapid and precise alignment of raw reads against redundant databases with KMA. *BMC Bioinformatics.* 2018;19(1):307. doi:10.1186/s12859-018-2336-6
3. Di Tommaso P, Chatzou M, Floden EW, Barja PP, Palumbo E, Notredame C. Nextflow enables reproducible computational workflows. *Nat Biotechnol.* 2017;35(4):316-319. doi:10.1038/nbt.3820
4. De Coster W, Rademakers R. NanoPack2: population-scale evaluation of long-read sequencing data. *Bioinformatics.* 2023;39(5):btad311. doi:10.1093/bioinformatics/btad311
5. Ewels P, Magnusson M, Lundin S, Käller M. MultiQC: summarize analysis results for multiple tools and samples in a single report. *Bioinformatics.* 2016;32(19):3047-3048. doi:10.1093/bioinformatics/btw354
6. Wick R. Filtrlong [software]. Disponible en: <https://github.com/rrwick/Filtrlong>

La referencia 1 corresponde a PubMLST, la 2 a KMA, la 3 a Nextflow, la 4 a NanoPlot y chopper, y la 5 a MultiQC. Filtrlong no tiene publicación asociada y se cita por su repositorio.

5.5 Componentes de terceros y fuente de datos

La lista completa de componentes de terceros, con sus versiones y licencias, está en [THIRD_PARTY_LICENSES.md](#).

Las definiciones de alelos y perfiles provienen de PubMLST (<https://pubmlst.org>), alojado en la Universidad de Oxford. Sus términos de uso exigen incluir el siguiente reconocimiento, en inglés y de forma literal, en toda publicación basada en sus datos, además de citar la referencia 1 de la sección 5.4:

This publication made use of the PubMLST website (<https://pubmlst.org/>) sited at the University of Oxford. The development of that website was funded by the Wellcome Trust.

6. Preguntas frecuentes

¿Debo instalar KMA, NanoPlot u otras herramientas por separado?

No. Cada proceso declara su propio ambiente Conda y Nextflow lo construye automáticamente en la primera corrida. Solo necesita Nextflow y Conda (sección 2.1).

¿Necesito una cuenta en PubMLST?

Sí, para obtener el esquema completo. La cuenta es gratuita; con ella se crea la clave de acceso que NAKAST usa (sección 2.4). Sin cuenta puede correr con `--pubmlst_anonymous`, pero solo con datos depositados hasta el 31 de diciembre de 2024.

¿Puedo analizar varias especies en una misma corrida?

Sí. Declare la especie de cada muestra en la columna `species`. NAKAST descarga una base por especie y genera un reporte por cada una.

¿Cómo sé qué nombre usar en la columna `species`?

Ejecute `nextflow run nakast.nf --list_species` y use el nombre corto exacto de la primera columna. El catálogo completo también está en `docs/supported_species.md`.

¿Por qué mi muestra tiene `ST = -`?

Casi siempre porque un locus quedó anotado (`~N`, `N?`, `INS` o `-`) y el perfil no puede coincidir con ninguno de PubMLST. La [sección 3.5](#) explica qué hacer en cada caso.

¿Qué significa que un alelo aparezca como `~9`?

Que el alelo más parecido es el 9, pero la secuencia no coincide exactamente. Puede tratarse de un alelo nuevo o de un error de secuenciación; si la diferencia es consistente a alta profundidad, confírmelo por Sanger.

Todas las muestras aparecen con ST = -. ¿Qué reviso primero?

El valor de especies. Tipificar contra el esquema equivocado produce resultados vacíos sin ningún error. Después revise las columnas de alelo.

¿Puedo usar datos Illumina?

No. NAKAST procesa solo lecturas Oxford Nanopore y sus parámetros están afinados para ese perfil de error.

¿Funciona sin conexión a internet?

No. Cada corrida descarga el esquema vigente desde PubMLST.

La descarga desde PubMLST falló. ¿Pierdo lo avanzado?

No. El pipeline reintenta tres veces, deja terminar el control de calidad en curso y se detiene. Cuando se restablezca la conexión, vuelva a ejecutar el mismo comando agregando `-resume`; el trabajo completado se reutiliza.

-resume no reutiliza nada. ¿Por qué?

Algo en las entradas de las tareas cambió entre corridas. Si modificó el código, tenga en cuenta que interpolar un objeto Path como `launchDir` directamente en el script de un proceso le da un hash inestable; debe pasarse como entrada `val`.

Falta una muestra en el reporte.

Su paso de filtrado o de control de calidad falló y se ignoró para que el resto del lote continuara. Busque `Error is ignored` en el log de Nextflow y revise `pipeline_info/` para ver qué proceso falló.

La validación rechaza la especie.

El nombre no está en el catálogo. Ejecute `--list_species` y use el nombre corto exacto; el mensaje de error sugiere coincidencias cercanas.

Aparece No PubMLST API key found.

No hay una clave guardada en el almacén de secretos de Nextflow. Guárdela como indica la sección 2.4, o agregue `--pubmlst_anonymous` para correr solo con datos hasta 2024.

Aparece PubMLST rejected the API key (HTTP 401).

La clave guardada no es válida o fue revocada. Cree una nueva en su cuenta de PubMLST y vuelva a guardarla con el comando de la sección 2.4; el valor anterior se reemplaza.

Aparece Unknown configuration profile: 'conda'.

Conda está habilitado por defecto. Quite el flag `-profile conda` del comando.

Aparece `command not found` o `exit status 127`.

El proceso corrió contra el PATH del sistema en vez de su ambiente Conda. Confirme que `nextflow config` . muestre `conda { enabled = true }` y que ningún `nextflow.config` en su directorio de lanzamiento lo esté sobrescribiendo.

Aparece `Variable declarations cannot be mixed with config statements`.

Un `nextflow.config` en su directorio de lanzamiento usa sintaxis que Nextflow 26 rechaza. Nextflow combina ese archivo con el del pipeline, así que un config ajeno de otro proyecto rompe la corrida. Lance desde un directorio limpio.

¿Cómo reproduzco exactamente una corrida antigua?

Conserve junto al reporte el archivo `reports/<especie>_profiles.tsv` y la traza de `pipeline_info/`. Las herramientas tienen versiones fijas; lo único que cambia con el tiempo es PubMLST, y ese archivo registra la versión usada.